

## 20 Testing Java persistence applications

This chapter covers

- Introducing the testing pyramid and examining persistence testing in its context
- Creating a persistence application to test using Spring Boot
- Using the Spring TestContext Framework
- Working with Spring profiles to test Java persistence applications
- Using execution listeners to test Java persistence applications

All code needs to be tested. During development, we code, compile, and run. When we run, we may sometimes test how our code works. Testing persistence applications involves more than this. In those cases, our code interacts with the external database, and how our program works may be dependent on it.

### 20.1 Introducing the test pyramid

In previous chapters, we focused on developing code that interacts with a database. We examined different alternatives for doing this, and different frameworks, and we interacted with various databases. Now we need to make sure that our programs are safe and bug-free. We need to be able to introduce changes without creating bugs, add new features without affecting the old features, and refactor code without breaking existing functionality. That is the purpose of this final chapter.

Applications can be tested manually, but nowadays most tests are executed automatically and they address different levels. The different levels of software tests for monolithic applications can be regarded as a pyramid, as shown in figure 20.1. We can define the following levels of software testing (from lowest to highest):

- *Unit testing*—Unit testing is at the foundation of the pyramid. It focuses on methods or classes (individual units) by testing each one in isolation to determine whether it works according to expectations.
- *Integration testing*—Individual, verified software components are combined in larger aggregates and tested together.

- *System testing*—Testing is performed on a complete system, to evaluate its compliance with the specification. System testing requires no knowledge of the design or code but focuses on the functionality of the entire system.
- *Acceptance testing*—Acceptance testing uses scenarios and test cases to check whether the application satisfies the expectations of the end user.

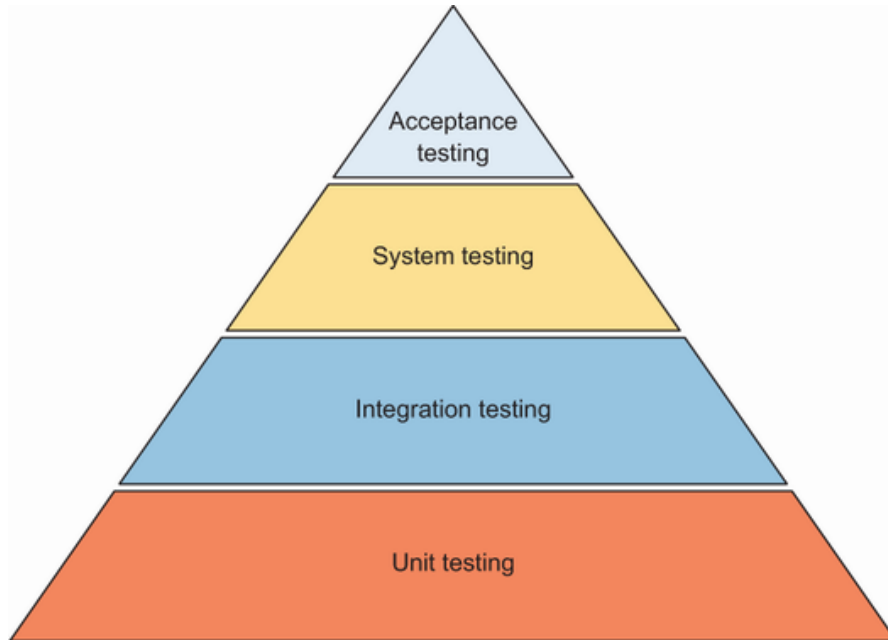


Figure 20.1 The testing pyramid has a larger bottom level (unit tests), while the higher testing levels are smaller. They start with checking individual units and go up to verifying how software addresses the user's needs.

Testing persistence applications belongs to the integration level. We combine our code with the database interaction, and we are dependent on how the database works. We would like to keep the behavior of our tests consistent between repeated test executions and to keep the content of the database the same as it was before running the tests. We'll examine the best ways to achieve these goals in the following sections.

## 20.2 Creating the persistence application to test

We'll create a Spring Boot persistence application so we can test its functionality. To do this, go to the Spring Initializr website

(<https://start.spring.io/>) and create a new Spring Boot project (figure 20.2), having the following characteristics:

- Group: com.manning.javapersistence
- Artifact: testing
- Description: Persistence Testing

We'll also add the following dependencies:

- Spring Data JPA (this will add `spring-boot-starter-data-jpa` in the Maven `pom.xml` file).
- MySQL Driver (this will add `mysql-connector-java` in the Maven `pom.xml` file).
- Lombok (this will add `org.projectlombok,lombok` in the Maven `pom.xml` file).
- Bean Validation with Hibernate validator (this will add `spring-boot-starter-validation` in the Maven `pom.xml` file).
- The `spring-boot-starter-test` dependency will be automatically added.

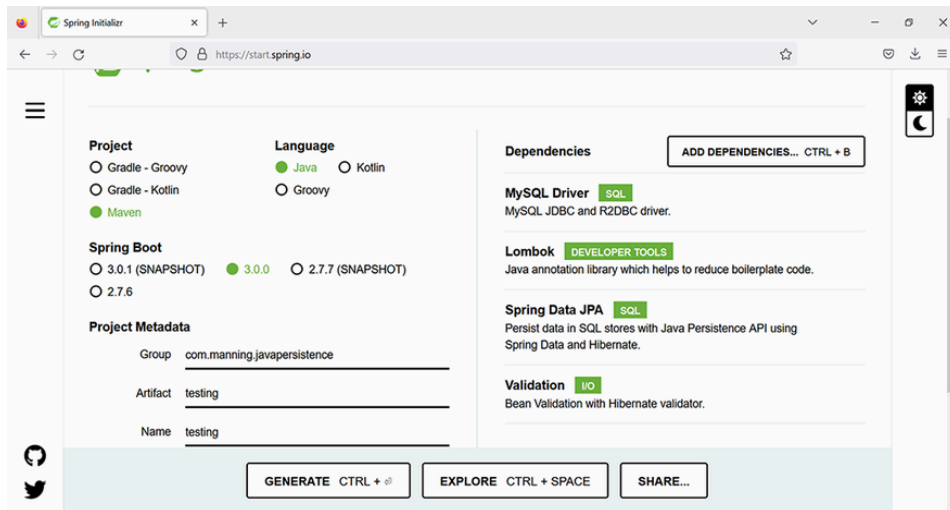


Figure 20.2 Creating a new Spring Boot project that will use a MySQL database through Spring Data JPA

**NOTE** To execute the examples from the source code, you'll first need to run the `Ch20.sql` script.

The `pom.xml` file in the following listing includes the dependencies that we added when we created the Spring Boot project. We'll create a Spring persistence application that accesses a MySQL database, for which we'll need the driver and we'll also need to validate some fields.

Listing 20.1 The `pom.xml` Maven file

Path: `Ch20/1 spring testing/pom.xml`

```
\1
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
```

```

        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>

```

- Ⓐ `spring-boot-starter-data-jpa` is the starter dependency used by Spring Boot to connect to a relational database through Spring Data JPA.
- Ⓑ `spring-boot-starter-validation` is the starter dependency for using Java Bean Validation with Hibernate Validator.
- Ⓒ `mysql-connector-java` is the JDBC driver for MySQL. It's a runtime dependency, so it's needed in the classpath only at runtime.
- Ⓓ `Lombok` will allow us to reduce the boilerplate code, relying on automatically generated constructors, getters, and setters.
- Ⓔ `spring-boot-starter-test` is the starter for testing Spring Boot applications with libraries including JUnit 5.

The next step is to fill in the Spring Boot `application.properties` file. This file can include various properties to be used by the application. Spring Boot will automatically find and load `application.properties` from the classpath; the `src/main/resources` folder is added by Maven to the classpath. For this chapter's demonstration, the `application.properties` configuration file will look like listing 20.2.

Listing 20.2 The `application.properties` file

```

Path: Ch20/1 spring testing/src/main/resources/application.properties

spring.datasource.url=jdbc:mysql://localhost:3306/
➡ CH20_TESTING?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create

```

- Ⓐ The URL of the database.

- Ⓑ The credentials to access the database. Replace them with the credentials on your machine, and use a password in real life.
- Ⓒ The dialect of the database, MySQL.
- Ⓓ Show the SQL queries while they are executed.
- Ⓔ Recreate the tables for each execution of the application.

**NOTE** There are several ways to provide parameters in a Spring Boot application, and the `.properties` file is just one of them. Among the alternatives, the parameters may come from the source code or as command-line arguments. Refer to the Spring Boot documentation for details.

The application that we'll test will include two entities, `User` and `Log`. The `User` entity is shown in the following listing. It will have a generated `id` and a `name` field with length validation. The no-arguments constructor, getters, and setters will be generated by Lombok.

Listing 20.3 The `User` class

```
Path: Ch20/1 spring testing/src/main/java/com/manning/javapersistence
➡ /testing/model/User.java
```

```
@Entity
@NoArgsConstructor
public class User {

    @Id
    @GeneratedValue
    @Getter
    private Long id;

    @NotNull
    @Size(
        min = 2,
        max = 255,
        message = "Name is required, maximum 255 characters."
    )
    @Getter
    @Setter
    private String name;

    public User(String name) {
        this.name = name;
    }

}
```

The `Log` entity is shown in the following listing. It will have a generated `id` and an `info` field with length validation. The no-arguments constructor, getters, and setters will be generated by Lombok.

Listing 20.4 The `Log` class

```
Path: Ch20/1 spring testing/src/main/java/com/manning/javapersistence
➡ /testing/model/Log.java
```

```
@Entity
@NoArgsConstructor
public class Log {

    @Id
    @GeneratedValue
    @Getter
    private Long id;

    @NotNull
    @Size(
        min = 2,
        max = 255,
        message = "Info is required, maximum 255 characters."
    )
    @Getter
    @Setter
    private String info;

    public Log(String info) {
        this.info = info;
    }
}
```

To manage the two entities, we'll create two repository interfaces to extend `JpaRepository`: `UserRepository` and `LogRepository`:

```
public interface UserRepository extends JpaRepository<User, Long> {
}

public interface LogRepository extends JpaRepository<Log, Long> {
}
```

## 20.3 Using the Spring TestContext Framework

The Spring TestContext Framework is designed to provide integration testing support, so it is ideal for testing the persistence layer. It is agnostic of the testing framework we use with it, and we'll work with JUnit 5. We'll

examine its essential classes and annotations, which belong to the `org.springframework.test.context` package.

The entry point into the Spring TestContext Framework is the `TestContextManager` class. Its goal is to manage a single `TestContext` and to send events to the registered listeners. We'll examine listeners in detail in section 20.8.

We'll write a first test that will save a `User` entity to the database and retrieve it again, using an injected `UserRepository`.

Listing 20.5 The `SaveRetrieveUserTest` class

```
Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/SaveRetrieveUserTest.java
```

```
@SpringBootTest
class SaveRetrieveUserTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void saveRetrieve() {
        userRepository.save(new User("User1"));
        List<User> users = userRepository.findAll();

        assertAll(
            () -> assertEquals(1, users.size()),
            () -> assertEquals("User1", users.get(0).getName())
        );
    }
}
```

If we repeatedly run this test several times, it will always succeed, so we might try to modify it and annotate the `saveRetrieve` test method with the `@RepeatedTest(2)` JUnit 5 annotation. This will run it twice in one execution of the class. The modified test will look like the following listing.

Listing 20.6 The `SaveRetrieveUserTest` class using `@RepeatedTest`

```
Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/SaveRetrieveUserTest.java
```

```
@SpringBootTest
class SaveRetrieveUserTest {

    @Autowired
```

```

private UserRepository userRepository;

@RepeatedTest(2)
void saveRetrieve() {
    userRepository.save(new User("User1"));
    List<User> users = userRepository.findAll();

    assertEquals(1, users.size());
    assertEquals("User1", users.get(0).getName());
};
}
}

```

Let's run the modified test now. Surprising or not, it will succeed for the first execution and will fail for the second, getting two `User`s from the database instead of one (see figure 20.3).

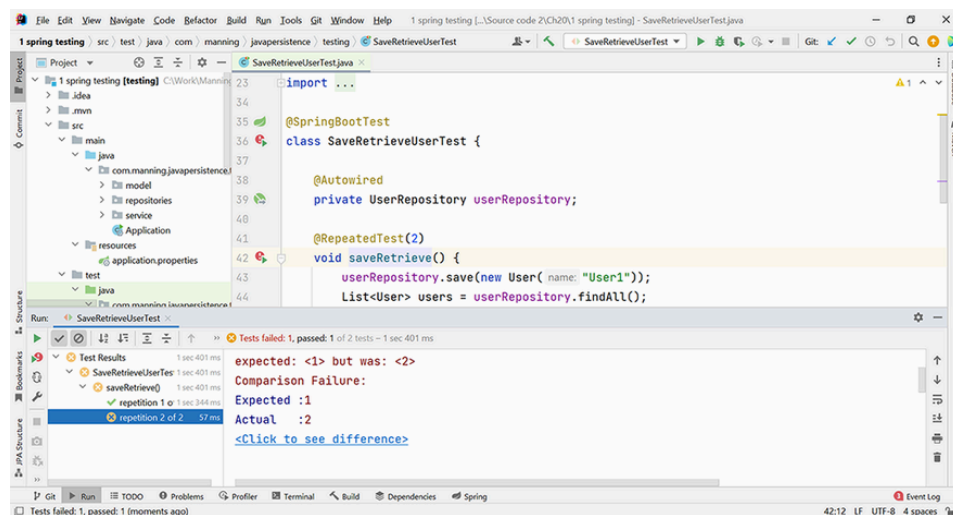


Figure 20.3 The `@RepeatedTest` will succeed the first time and will fail the second time, as the database was left dirty after the execution of the first test.

This happens because the row that was inserted by the execution of the first test was not removed, and the second test found it in the table, adding one more. We can follow the SQL commands that are executed when the class is run.

Before running the two tests, the SQL commands that are executed deal with the (re)creation of tables:

```

drop table if exists hibernate_sequence
drop table if exists log
drop table if exists user
create table hibernate_sequence (next_val bigint)
insert into hibernate_sequence values ( 1 )
create table log (id bigint not null, info varchar(255) not null,
➡ primary key (id))

```



```
create table user (id bigint not null, name varchar(255) not null,
➡ primary key (id))
```

Before running each test, the following SQL commands are executed, inserting a new row into the table:

```
select next_val as id_val from hibernate_sequence for update
update hibernate_sequence set next_val= ? where next_val=?
insert into user (name, id) values (?, ?)
select user0_.id as id1_1_, user0_.name as name2_1_ from user user0_
```

The execution of the second test will find an existing row in the table, will add a new one, and so will fail because it expects to find a single row. We'll have to look for alternatives to leave the content of the database at the end of each test the same as it was before running it.

## 20.4 The @DirtiesContext annotation

One alternative, provided by the Spring TestContext Framework, is to use the @DirtiesContext annotation. @DirtiesContext acknowledges that the test method or the test class changes the Spring context, and the Spring TestContext Framework will recreate it from scratch and provide it to the next test. The annotation can be applied to a method or a class. Its effects can be applied before or after the execution of each test method, or before or after the execution of the test class.

We'll modify the test class as shown in the following listing.

Listing 20.7 The SaveRetrieveUserTest class using @DirtiesContext

```
Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/SaveRetrieveUserTest.java

@SpringBootTest
@DirtiesContext(classMode =
➡ DirtiesContext.ClassMode.AFTER_EACH_TEST_METHOD) ①
class SaveRetrieveUserTest {

    @Autowired
    private UserRepository userRepository;

    @RepeatedTest(2)
    void saveRetrieve() {
        // . . .
    }

}
```

Ⓐ Recreate the Spring context after the execution of each test method.

If we run the modified test now, it will succeed for the first and second executions (see figure 20.4). This means that after running the first test, the execution of the second test will no longer encounter a dirtied database.

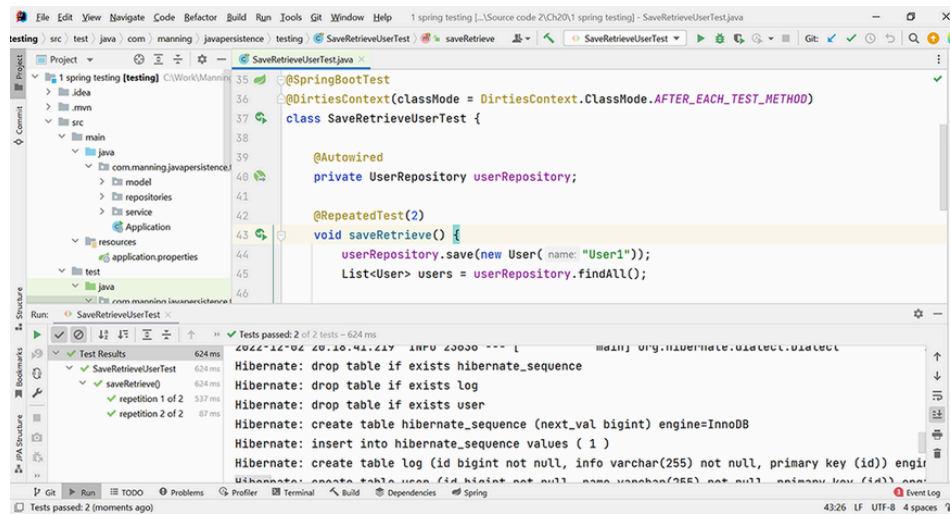


Figure 20.4 The `@DirtyContext` annotated test will succeed both the first and second time.

Before running each test, the SQL commands that are executed deal with both the (re)creation of tables and inserting a row in the table:

```
drop table if exists hibernate_sequence
drop table if exists log
drop table if exists user
create table hibernate_sequence (next_val bigint)
insert into hibernate_sequence values ( 1 )
create table log (id bigint not null, info varchar(255) not null,
➡ primary key (id))
create table user (id bigint not null, name varchar(255) not null,
➡ primary key (id))
select next_val as id_val from hibernate_sequence for update
update hibernate_sequence set next_val= ? where next_val=?
insert into user (name, id) values (?, ?)
select user0_.id as id1_1_, user0_.name as name2_1_ from user user0_
```

These commands are executed twice, once before each test. More than this, the Spring Boot banner (also displayed at the bottom of figure 20.4) will be shown twice as well—once each time the application is started.

The functionality of the `@DirtyContext` annotation at the method level is demonstrated in figure 20.5.

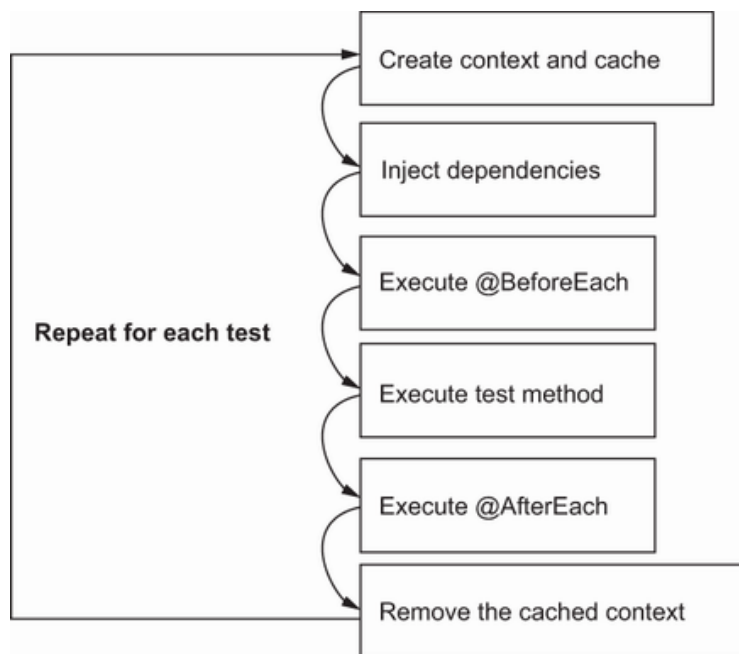


Figure 20.5 The `@DirtyContext` annotation at the method level will create the context and cache before the execution of each test method and will remove them after the execution.

This solution works, but it comes with a performance cost related to the recreation of tables and reinitialization of the application for each test execution. Let's explore more alternatives.

## 20.5 @Transactional execution

We examined transactions in detail in chapter 11. Transactions control atomic groups of operations that either fully succeed or fully fail. Managing transactions with Spring and Spring Data and the `@Transactional` annotation were detailed in section 11.4. The idea that we'll apply now is to run each test transactionally and to roll back the transaction at the end of the execution.

By default, when executing a test, our transactions will automatically be rolled back due to the `TransactionalTestExecutionListener`. We'll deal in detail with listeners in section 20.8; for now, just note that they can provide some additional action at the execution of a test. The default behavior can be modified with the help of the `@Commit` and `@Rollback` annotations. So if we want a test to commit at the end of the execution, we can annotate it either with `@Commit` or with `@Rollback(false)`.

To keep track of when a transaction is active during the execution of a test, we'll use the `TransactionSynchronizationManager` class. This class manages resources and transaction synchronizations for a thread. Its `isActualTransactionActive()` method will check whether a `Transaction` object is currently active.

In the following listing, we'll create a test class, annotate it with `@Transactional`, and follow the status of the transaction inside the methods of the class.

```

Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/TransactionalTest.java

@SpringBootTest
@Transactional
class TransactionalTest {

    // . . .

    @BeforeAll
    static void beforeAll() {
        System.out.println("beforeAll, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive()
        )
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("beforeEach, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive()
        )
    }

    @RepeatedTest(2)
    void storeRetrieve() {
        // . . .
        System.out.println("end of method, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive()
        )
    }

    @AfterEach
    void afterEach() {
        System.out.println("afterEach, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive()
        )
    }

    @AfterAll
    static void afterAll() {
        System.out.println("afterAll, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive()
        )
    }

}

```

The log of the execution will tell us that the transaction is not active in the `@BeforeAll` and `@AfterAll` methods, but it is active in the `@BeforeEach` and `@AfterEach` methods and inside the test method itself. As previously stated, the transaction will be rolled back by default at the end of the test, so `@RepeatedTest` will succeed for all executions:

```

beforeAll, transaction active = false
beforeEach, transaction active = true
end of method, transaction active = true
afterEach, transaction active = true
beforeEach, transaction active = true
end of method, transaction active = true
afterEach, transaction active = true
afterAll, transaction active = false

```

There is still a pitfall of this approach that we will demonstrate. We introduced a separate `UserService` class with a transactional method, as shown in the following listing.

Listing 20.9 The `UserService` class

```

Path: Ch20/1 spring testing/src/main/java/com/manning/javapersistence
➡ /testing/service/UserService.java

```

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Transactional
    public void saveTransactionally(User user) {
        userRepository.save(user);
    }
}

```

We'll call this method from inside the `@RepeatedTest` from the `TransactionalTest` class, to persist a user in a transactional way.

Listing 20.10 Calling the `saveTransactionally` method

```

Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/TransactionalTest.java

```

```

@SpringBootTest
@Transactional
class TransactionalTest {

    // . . .

    @RepeatedTest(2)
    void storeRetrieve() {
        List<User> users = buildUsersList();
        userRepository.saveAll(users);
        assertEquals(getIterations(), userRepository.findAll().size());
    }
}

```

```

userService.saveTransactionally(users.get(0));

System.out.println("end of method, transaction active = " +
    TransactionSynchronizationManager.isActualTransactionAct
}

// . . .

}

```

The `saveTransactionally` method from `UserService` has the annotation `@Transactional` with no other argument. The default propagation is `REQUIRED` (see section 11.4). As there is already a transaction running for the test, the `saveTransactionally` method will execute within the same transaction, and everything will be rolled back at the end of the test.

We can change the `saveTransactionally` method annotation to be `@Transactional(propagation = Propagation.REQUIRES_NEW)`. This will suspend the transaction executed in the test, start a new transaction, and commit it, as figure 20.6 shows.

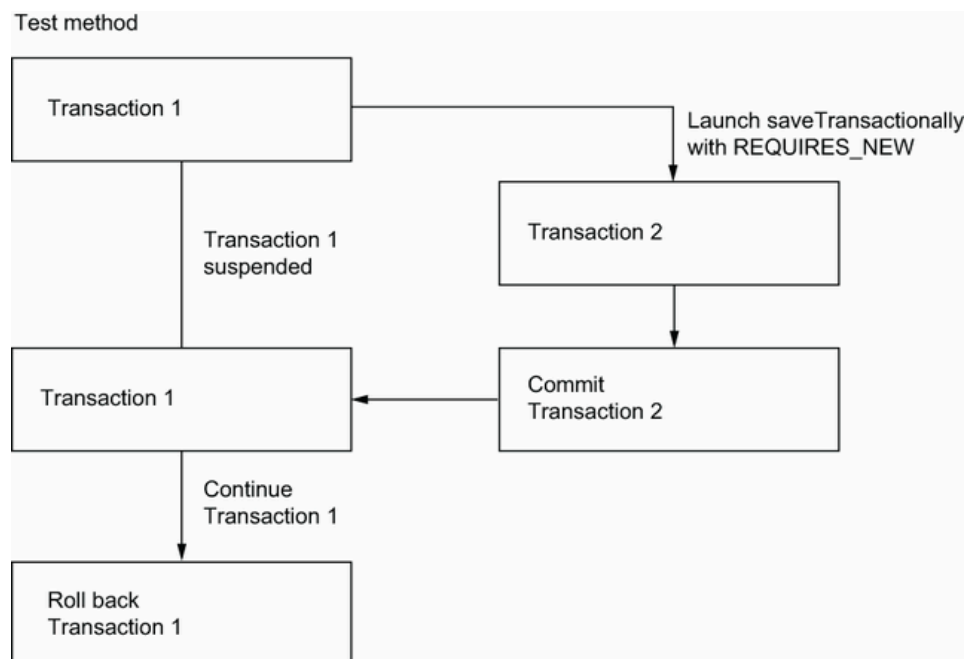


Figure 20.6 The transaction from the `saveTransactionally` method will be committed, and the transaction from the test method will be rolled back.

The transaction from `saveTransactionally` was committed. As a consequence, running the test for a second time will encounter an existing record in the database, and the test will fail (see figure 20.7).

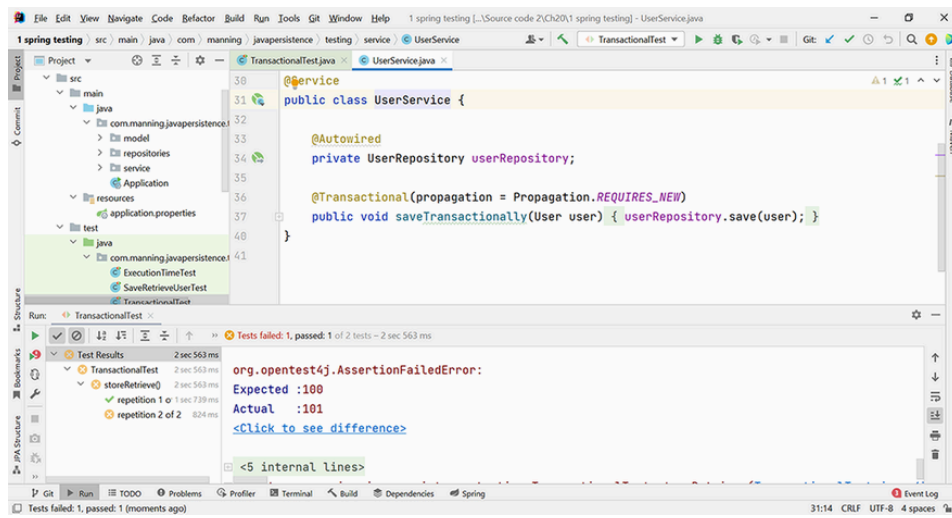


Figure 20.7 Running two tests in a row while the `saveTransactionally` method commits its transaction separately will make the second test fail.

The conclusion is that even if you run your tests transactionally, beware of the pitfall of launching methods in separate transactions. It can lead to strange bugs. Also, debugging might be difficult with this approach.

To make a performance comparison between using `@DirtyiesContext` and using `@Transactional`, we executed a series of 10 tests, progressively increasing the number of records from 100 to 2,000. The results on MySQL are provided in figure 20.8.

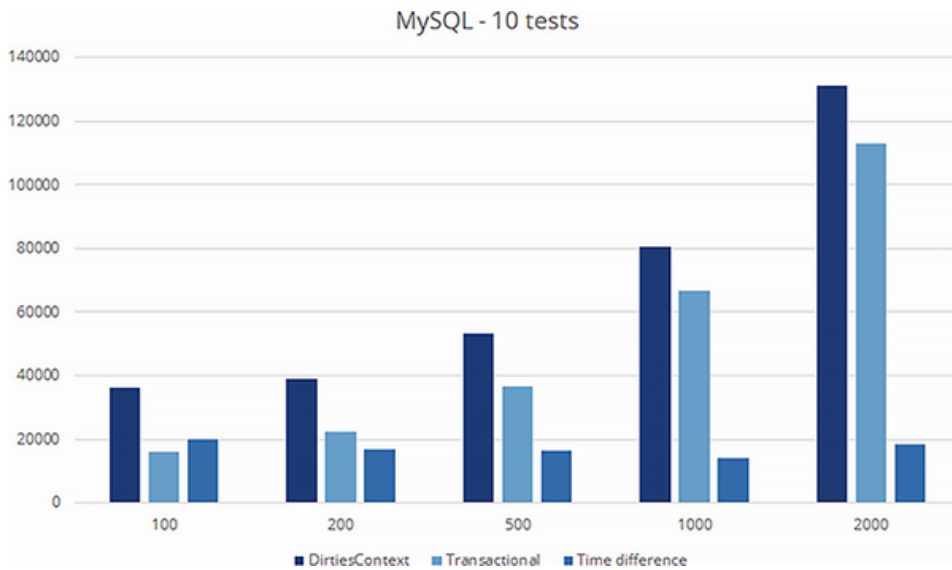


Figure 20.8 The execution times (in ms) on MySQL using `@DirtyiesContext` and `@Transactional` and varying the number of records from 100 to 2,000

The results on H2 are provided in figure 20.9. We executed the same series of 10 tests using this in-memory database, progressively increasing the number of records from 100 to 2,000.

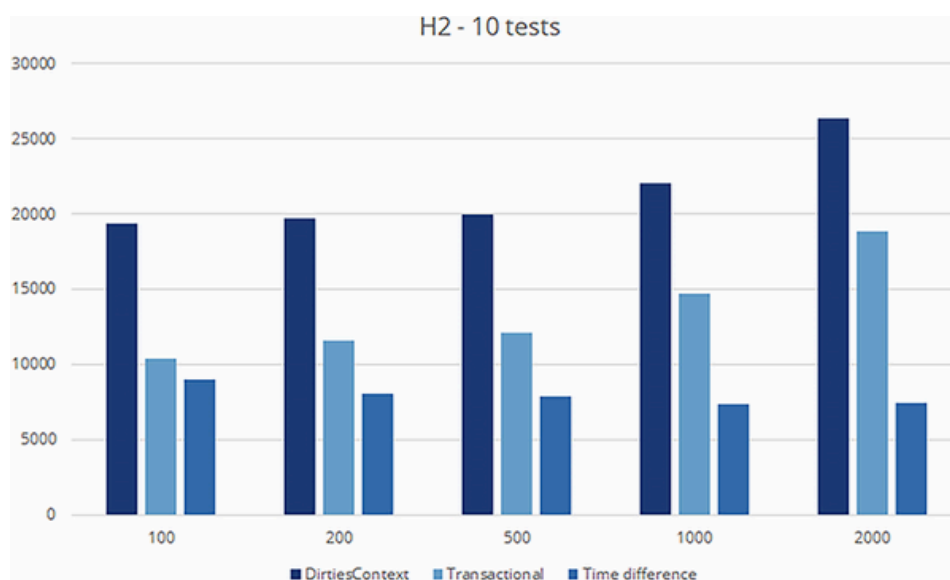


Figure 20.9 The execution times (in ms) on H2 using `@DirtyiesContext` and `@Transactional` and varying the number of records from 100 to 2,000

Analyzing the results for both MySQL and H2, we can see that the difference between the execution with `@DirtyiesContext` and with `@Transactional` is approximately constant. It does not depend on the number of records, but on the number of times the context is reinitialized. The conclusion is that you should use `@DirtyiesContext` sparingly. Pushing tests with this annotation to the CI/CD (continuous integration/continuous development) environment will seriously increase the time of their execution.

## 20.6 The `@BeforeTransaction` and `@AfterTransaction` annotations

We'll now examine the `@BeforeTransaction` and `@AfterTransaction` annotations. As their names suggest, they indicate methods to be executed before and after the execution of a transaction. For our analysis, we'll check that, indeed, there is no transaction active inside such a method.

Using the `Assumptions.assumeFalse` JUnit 5 method, we'll indicate that a precondition for running the tests is that there is no transaction active at that moment; otherwise the tests will not run. So if an assumption is not met, the test will be aborted. If an assertion is not met, the test will fail.

Listing 20.11 Using `@BeforeTransaction` and `@AfterTransaction`

```
Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence
➡ /testing/TransactionsManagementTest.java
```

```
@SpringBootTest
@Transactional
class TransactionsManagementTest {
```



```

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private LogRepository logRepository;

    @BeforeTransaction
    void beforeTransaction() {
        Assumptions.assumeFalse(
            TransactionSynchronizationManager.isActualTransactionActive()
        );
    }

    // . . .

    @AfterTransaction
    void afterTransaction() {
        Assumptions.assumeFalse(
            TransactionSynchronizationManager.isActualTransactionActive()
        );
    }
}

```

There is still one pitfall to avoid here: the possibility of persisting data in the `@BeforeTransaction` or `@AfterTransaction` methods. Because these are executed outside transactions, data will not be rolled back and will affect the content of the database. Even more, if we persist data that we do not check in the tests (for example, `Log` entities, while our tests verify `User` entities), our tests will always execute correctly but will leave committed data behind them, as happens in the following listing.

Listing 20.12 Persisting entities in `@BeforeTransaction` / `@AfterTransaction`

Path: Ch20/1 spring testing/src/test/java/com/manning/javapersistence  
 ➡ /testing/TransactionsManagementTest.java

```

@SpringBootTest
@Transactional
class TransactionsManagementTest {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private LogRepository logRepository;

    @BeforeTransaction
    void beforeTransaction() {
        Assumptions.assumeFalse(

```

```

        TransactionSynchronizationManager.isActualTransactionActive();
        logRepository.save(new Log("@BeforeTransaction"));
    }

    // . . .

    @AfterTransaction
    void afterTransaction() {
        Assumptions.assumeFalse(
            TransactionSynchronizationManager.isActualTransactionActive());
        logRepository.save(new Log("@AfterTransaction"));
    }
}

```

## 20.7 Working with Spring profiles

By default, Spring Boot creates a `main/resources/application.properties` file to keep the configuration of the application. But there are frequent situations where we'll need to differentiate between properties depending on a user's profile. Spring Boot will allow us to separate the properties in this case, in files named, by default, `main/resources/application-profilename.properties`, and it will allow us to switch between profiles.

A real-life use case would be when you have a profile for programmers, using an embedded database during development, and another profile for production, using a real database. Programmers would like to be able to quickly execute their tests, while in production the tests will run in the real environment.

The configuration for development is shown in the following listing. It addresses an H2 database and will display SQL queries during execution, because the programmer is interested in following them.

Listing 20.13 The `application-dev.properties` file

```

Path: Ch20/2 spring profiles/src/main/resources
➡ /application-dev.properties

spring.datasource.url=jdbc:h2:mem:ch20_testing
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.H2Dialect
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=create

```

The configuration for production is shown in the following listing. It addresses a MySQL database and will not display the SQL queries during ex-

ecution, as this would consume resources in production.

Listing 20.14 The application-prod.properties file

```
Path: Ch20/2 spring profiles/src/main/resources
➡ /application-prod.properties

spring.datasource.url=
➡ jdbc:mysql://localhost:3306/CH20_TESTING?serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.show-sql=false
spring.jpa.hibernate.ddl-auto=create
```

To run the tests on the H2 database during development, we'll have to choose the `dev` profile. This can be done, for example, from within the `application.properties` file.

Listing 20.15 The application.properties file with the dev profile

```
Path: Ch20/2 spring profiles/src/main/resources/application.properties

spring.profiles.active=dev
```

To demonstrate how to easy it is to switch between profiles, we'll run one test that will save and retrieve an entity from the database.

Listing 20.16 The `SpringProfilesTest` class

```
Path: Ch20/2 spring profiles/src/test/java/com/manning/javapersistence
➡ /testing/SpringProfilesTest.java

@SpringBootTest
@Transactional
class SpringProfilesTest {

    @Autowired
    private UserRepository userRepository;

    @Test
    void storeUpdateRetrieve() {
        List<User> users = buildUsersList();
        userRepository.saveAll(users);

        assertEquals(getIterations(), userRepository.findAll().size());
    }
}
```

To successfully run this test, we'll need to have the H2 driver dependency in the pom.xml file.

Listing 20.17 The pom.xml file with the H2 driver dependency

Path: Ch20/2 spring profiles/pom.xml

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>1.4.200</version>
  <scope>runtime</scope>
</dependency>
```

The result of running the test on the `dev` profile is shown in figure 20.10: the `dev` profile uses the in-memory H2 database.

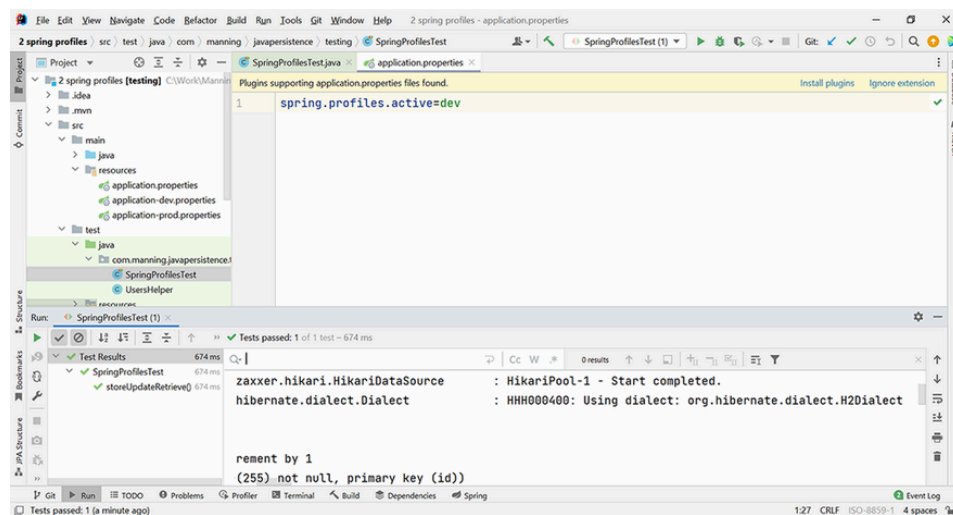


Figure 20.10 The result of running the test on the `dev` profile, using an H2 database and displaying the execution of the SQL queries

To run the tests on the MySQL database in production, we'll have to choose the `prod` profile. This can be done, for example, from within the application.properties file.

Listing 20.18 The application.properties file with the prod profile

Path: Ch20/2 spring profiles/src/main/resources/application.properties

```
spring.profiles.active=prod
```

As an alternative to modifying the active profile, we can use the `@ActiveProfiles` annotation at the level of the test, as shown in listing 20.19. This annotation will override the profile that is set in application.properties, but it will require us to modify and recompile the code.

## Listing 20.19 The SpringProfilesTest class

```
Path: Ch20/2 spring profiles/src/test/java/com/manning/javapersistence
➡ /testing/SpringProfilesTest.java

@SpringBootTest
@Transactional
@ActiveProfiles("prod")
class SpringProfilesTest {
    // . . .
}
```

To successfully run this test, we'll need to have the MySQL driver dependency in the pom.xml file.

## Listing 20.20 The pom.xml file with the MySQL driver dependency

```
Path: Ch20/2 spring profiles/pom.xml

<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <scope>runtime</scope>
</dependency>
```

The result of running the test on the `prod` profile is shown in figure 20.11. Unlike the `dev` profile, this profile uses the MySQL database.

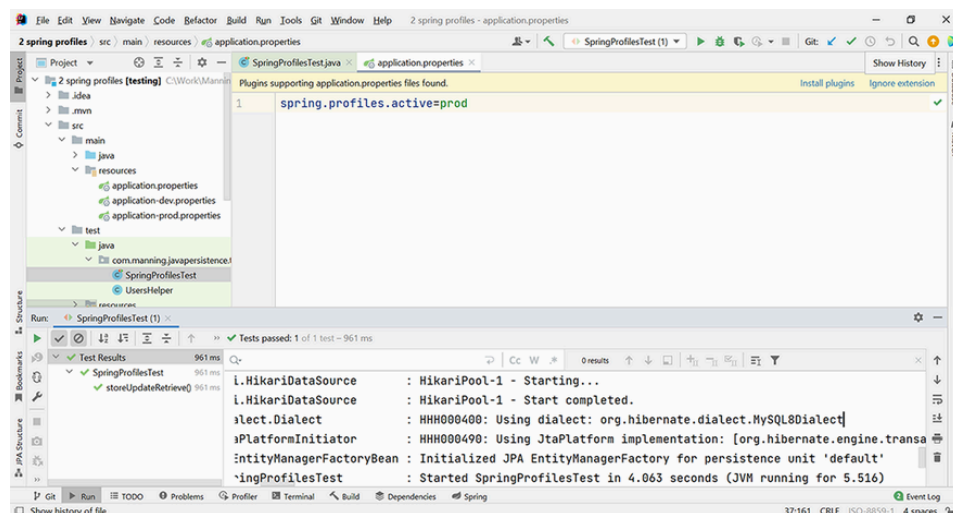


Figure 20.11 The result of running the test on the `prod` profile, using a MySQL database and not displaying the execution of the SQL queries

## 20.8 Working with test execution listeners

One way to control the lifecycle of a test's execution is to work with the JUnit 5 annotations: `@BeforeAll`, `@AfterAll`, `@BeforeEach`, and `@AfterEach`. This may be inconvenient in some situations. For example,

if we need the same `@BeforeEach` and `@AfterEach` behavior for several tests, we would need to create a base class containing these methods and to create multiple subclasses that will inherit and execute them when running the tests. This has the inconvenience of hanging our tests in a hierarchy of classes. Alternatively, we can consider working with test execution listeners, with the `TestExecutionListener` interface, and with the `@TestExecutionListeners` annotation, thus separating the behavior that controls the lifecycle of the test.

By default, Spring provides some already implemented `TestExecutionListener`s for each test. The ones of most interest to us here are `DependencyInjectionTestExecutionListener`, which supports dependency injection for the test instance, and `TransactionalTestExecutionListener`, which supports the transactional execution of a test with rollback. We mentioned in section 20.3.2 that, by default, when executing a test, the transaction will automatically be rolled back due to the `TransactionalTestExecutionListener` — this is essential when testing persistence and striving to leave a clean database after executing the tests.

The `TestExecutionListener` interface defines a series of empty default methods that are more fine-grained than the JUnit 5 lifecycle methods, and that are executed in the order shown in table 20.1.

Table 20.1 The default methods from the `TestExecutionListener` interface

| Method                             | Description                                                    |
|------------------------------------|----------------------------------------------------------------|
| <code>beforeTestClass()</code>     | Executed before the <code>@BeforeAll</code> method of JUnit 5  |
| <code>prepareTestInstance()</code> | Prepares the test instance of the supplied test context        |
| <code>beforeTestMethod()</code>    | Executed before the <code>@BeforeEach</code> method of JUnit 5 |
| <code>beforeTestExecution()</code> | Executed before the test method                                |
| <code>afterTestExecution()</code>  | Executed after the test method                                 |
| <code>afterTestMethod()</code>     | Executed after the <code>@AfterEach</code> method of JUnit 5   |
| <code>afterTestClass()</code>      | Executed after the <code>@AfterAll</code> method of JUnit 5    |

We'll write our own listener that implements the `TestExecutionListener` interface, override all its methods, and print a message from each, so we can follow the execution of the test that we'll annotate using this listener.

As in section 20.3.2, we'll follow when a transaction is active during the execution of a test by using the `TransactionSynchronizationManager` class and its method `isActualTransactionActive()`, which will check if a `Transaction` object is currently active. Our listener is shown in the following listing.

Listing 20.21 The `DatabaseOperationsListener` class

```
Path: Ch20/3 spring listeners/src/test/java/com/manning/javapersistence
    ➤ /testing/listeners/DatabaseOperationsListener.java

public class DatabaseOperationsListener implements TestExecutionListene

    @Override
    public void beforeTestClass(TestContext testContext) {
        System.out.println("beforeTestClass, transaction active = " +
```

```

        TransactionSynchronizationManager.isActualTransactionActive
    }

    @Override
    public void afterTestClass(TestContext testContext) {
        System.out.println("afterTestClass, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }

    @Override
    public void beforeTestMethod(TestContext testContext) {
        System.out.println("beforeTestMethod, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }

    @Override
    public void afterTestMethod(TestContext testContext) {
        System.out.println("afterTestMethod, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }

    @Override
    public void beforeTestExecution(TestContext testContext) {
        System.out.println("beforeTestExecution, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }

    @Override
    public void afterTestExecution(TestContext testContext) {
        System.out.println("afterTestExecution, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }

    @Override
    public void prepareTestInstance(TestContext testContext) {
        System.out.println("prepareTestInstance, transaction active = " +
            TransactionSynchronizationManager.isActualTransactionActive
        )
    }
}

```

We'll create our own test, annotate it to use the new `DatabaseOperationsListener`, and print messages from the lifecycle methods and the test itself, to follow its execution.

Listing 20.22 The `ListenersTest` class

```

Path: Ch20/3 spring listeners/src/test/java/com/manning/javapersistence
➡ /testing/ListenersTest.java

```

```

@SpringBootTest
@Transactional

```



```

@TestExecutionListeners(value = {DatabaseOperationsListener.class})
class ListenersTest {

    @Autowired
    private UserRepository userRepository;

    @BeforeAll
    static void beforeAll() {
        System.out.println("@BeforeAll");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("@BeforeEach");
    }

    @Test
    void storeUpdateRetrieve() {
        TestContextManager testContextManager = new
            TestContextManager(getClass());

        System.out.println(
            "testContextManager.getTestExecutionListeners().size() = "
            + testContextManager.getTestExecutionListeners().size());
        List<User> users = buildUsersList();
        userRepository.saveAll(users);
        assertEquals(getIterations(), userRepository.findAll().size());
    }

    @AfterEach
    void afterEach() {
        System.out.println("@AfterEach");
    }

    @AfterAll
    static void afterAll() {
        System.out.println("@AfterAll");
    }
}

```

If we run this test now, it will fail with `NullPointerException`, as in figure 20.12.

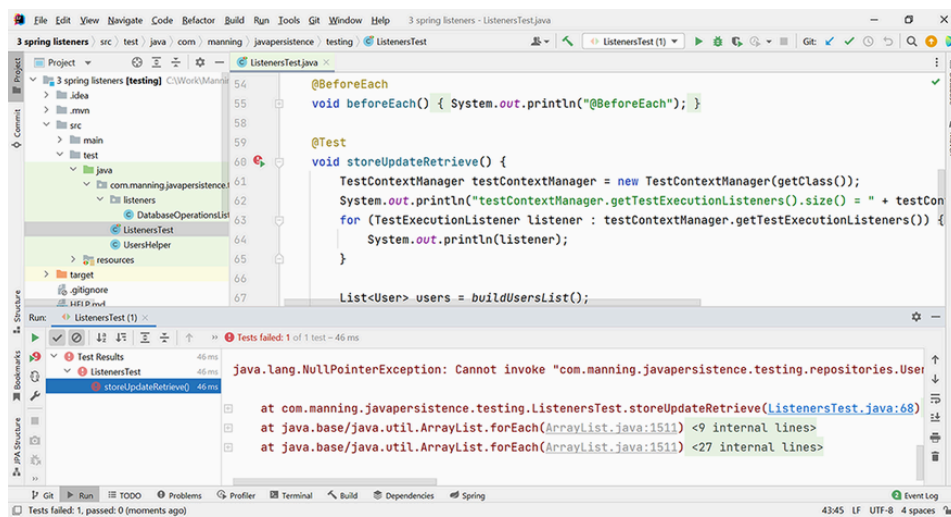


Figure 20.12 The test initially annotated with our custom listener fails with `NullPointerException`.

If we examine what is happening, we'll notice that the `userRepository` reference to the object that interacts with the database is `null`. As the messages from the console demonstrate, there is only one listener registered in the `TestContextManager`, our `DatabaseOperationsListener`. The previously needed `DependencyInjectionTestExecutionListener`, which supports dependency injection for the test instance, and consequently for the `userRepository`, is no longer registered. This is because, once we introduce our own listeners, the default listeners are no longer automatically registered.

To fix this, we'll use the `MERGE_WITH_DEFAULTS` option as merge mode, as follows.

#### Listing 20.23 Merging the default listeners with our custom listener

```
Path: Ch20/3 spring listeners/src/test/java/com/manning/javapersistence
  ➡ /testing/ListenersTest.java
```

```
@SpringBootTest
@Transactional
@TestExecutionListeners(value = {
    DatabaseOperationsListener.class}, mergeMode =
    TestExecutionListeners.MergeMode.MERGE_WITH_DEFAULTS)
class ListenersTest {
    // . . .
}
```

If we run the test again now, we'll be able to follow the sequence of the methods' execution, both from the listener and from the lifecycle methods of JUnit 5. We'll also notice that the number of registered listeners is now 15, meaning 14 default listeners and our custom listener:

```
beforeTestClass, transaction active = false
@BeforeAll
prepareTestInstance, transaction active = false
beforeTestMethod, transaction active = true
@BeforeEach
beforeTestExecution, transaction active = true
testContextManager.getTestExecutionListeners().size() = 15
afterTestExecution, transaction active = true
@AfterEach
afterTestMethod, transaction active = true
@AfterAll
afterTestClass, transaction active = false
```

Table 20.2 summarizes the most important annotations from the Spring TestContext Framework, to be used in testing persistence applications. You will be able to use these annotations from the Spring TestContext Framework for your persistence applications. This framework provides strong support for integration testing, to which persistence application testing belongs. On this base, you can continue to build system tests and acceptance tests, as we demonstrated at the beginning of this chapter when we introduced the testing pyramid. For more information about testing Java applications in general and about acceptance tests in particular, you can refer to my book *JUnit in Action*, third edition (Tudose, 2020).

Table 20.2 The most important Spring TestContext Framework annotations to be used in testing persistence applications

| Annotation                           | Description                                                                                                                                                                                          |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>@DirtyContext</code>           | The underlying Spring context was changed during the execution of a test and should be reinitialized.                                                                                                |
| <code>@BeforeTransaction</code>      | The void method should be executed before any method having the Spring <code>@Transactional</code> annotation.                                                                                       |
| <code>@AfterTransaction</code>       | The void method should be executed after any method having the Spring <code>@Transactional</code> annotation.                                                                                        |
| <code>@Rollback</code>               | The transaction for a transactional test will be rolled back after the test completes. This is the default behavior and may be changed using <code>@Rollback(false)</code> or <code>@Commit</code> . |
| <code>@Commit</code>                 | The transaction for a transactional test will be committed after the test completes.                                                                                                                 |
| <code>@ActiveProfiles</code>         | Specifies which configuration profiles will be active in the Spring context.                                                                                                                         |
| <code>@TestExecutionListeners</code> | Configures the test execution listeners to be registered with the <code>TestContextManager</code> .                                                                                                  |

With predictable and safe tests that fail only when there are problems in your code and not due to external factors (such as inappropriate content in a dirtied database), your life as a programmer will be much better!

## Summary

- The testing pyramid consists of the unit, integration, system, and acceptance levels. Persistence testing can be classified at the integration level.
- You can create and configure a persistence application using Spring Boot and manage entities and repositories inside it.
- You can use the Spring TestContext Framework to create persistence tests and manage them using either `@DirtyContext` or `@Transactional`.
- You can use Spring profiles to test Java persistence applications that access various databases and have different configurations.

- You can create a custom test execution listener to follow the lifecycle of a test and work with it and with the default listeners.